

CoLA: Compute-Efficient Pre-Training of LLMs via Low-Rank Activation

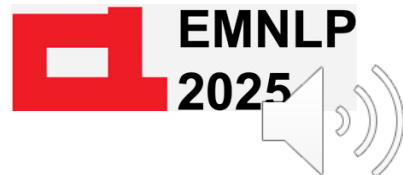
Ziyue Liu¹, Ruijie Zhang¹, Zhengyang Wang¹, Mingsong Yan¹, Zi Yang², Paul Hovland³,
Bogdan Nicolae³, Franck Cappello³, Sui Tang¹, Zheng Zhang¹

¹University of California, Santa Barbara

²University at Albany, SUNY

³Argonne National Laboratory

UC SANTA BARBARA



LLM Pre-Training

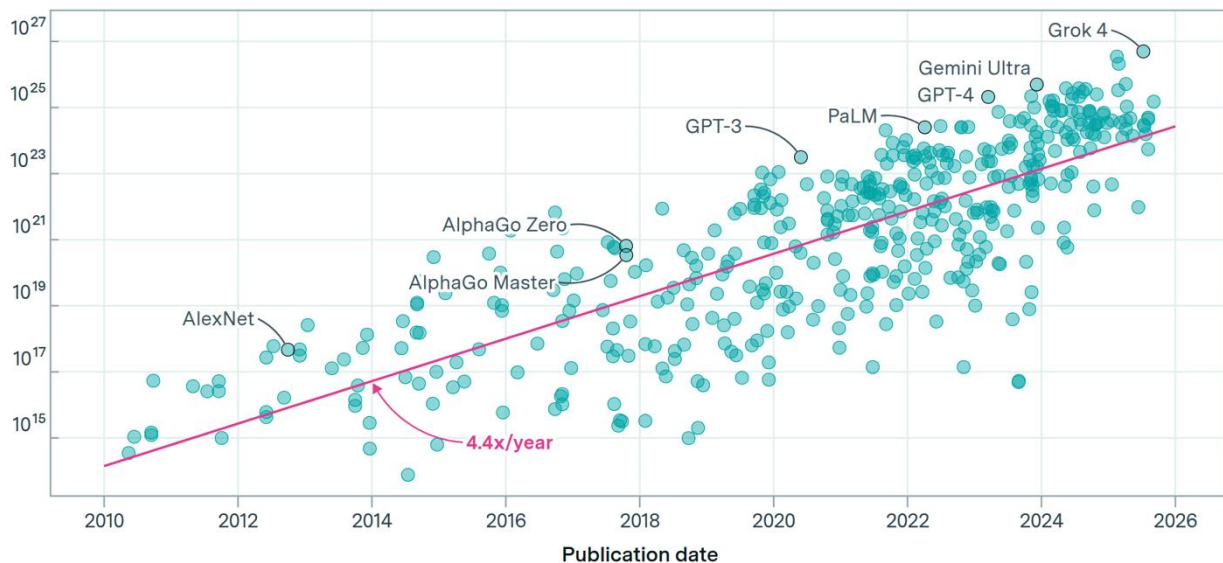
Massive compute, model size, memory, and training duration

Training compute of notable models

EPOCH AI

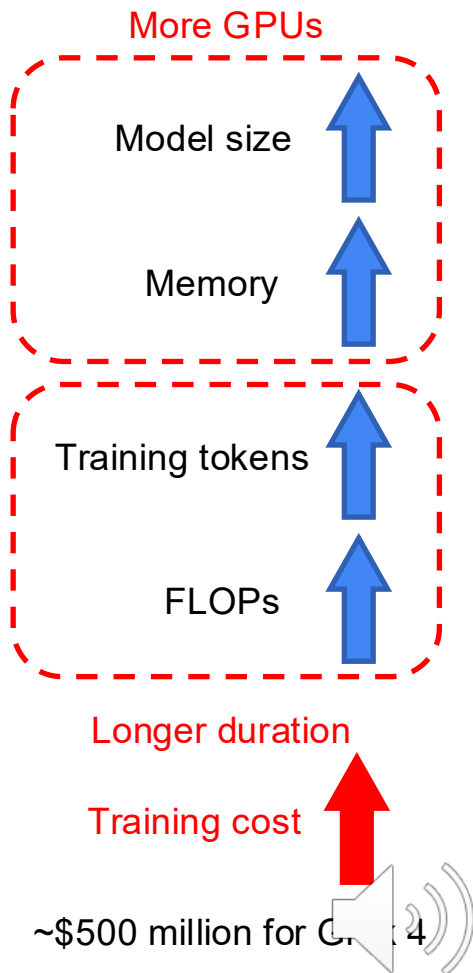
Training compute (FLOP)

443 models



CC-BY

epoch.ai



Existing Solutions

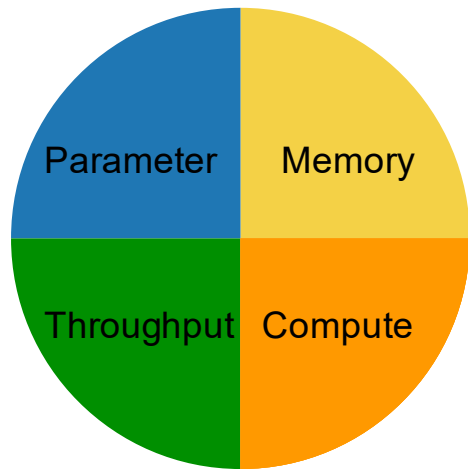
Improves **some but not all** from parameter ↓, memory ↓, compute ↓, and throughput ↑

GaLore (ICML 2024)

ReLoRA (ICLR 2024)

SLTrain (NeurIPS 2024)

CoLA (Ours) reduces parameter, compute, memory, and increases throughput at both training and inference stage

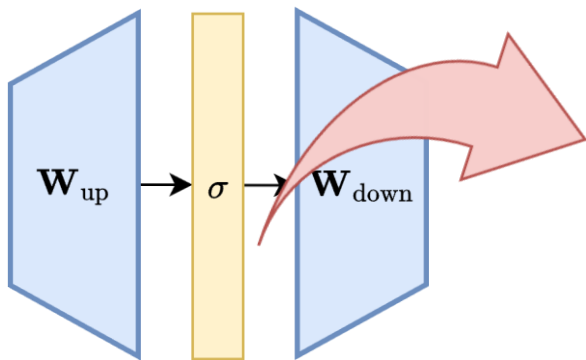


		GaLore	ReLoRA	SLTrain	CoLA(-M)
Parameter ↓		×	×	✓	✓
Compute ↓	Training	×	✓	×	✓
	Inference	×	×	×	✓
Memory ↓	Training	✓	✓	✓	✓
	Inference	×	×	✓	✓
Throughput ↑	Training	×	×	×	✓
	Inference	×	×	×	✓

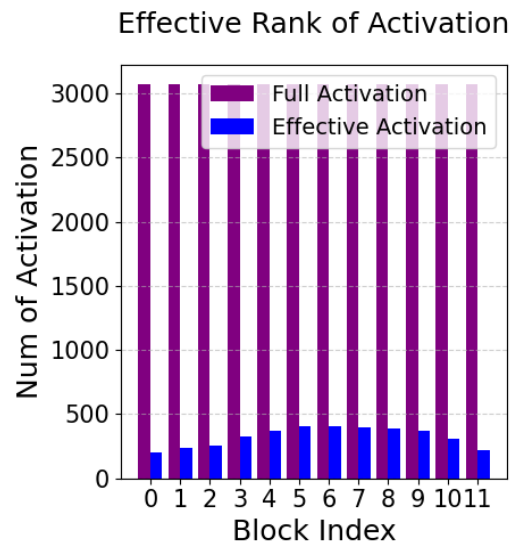
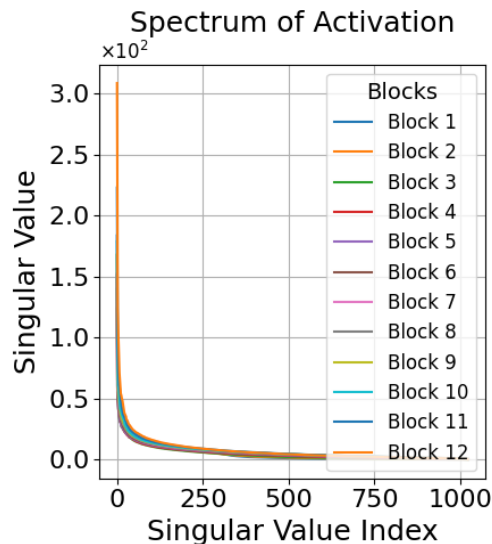


Observations

- **Activation function placed at the largest dimension**

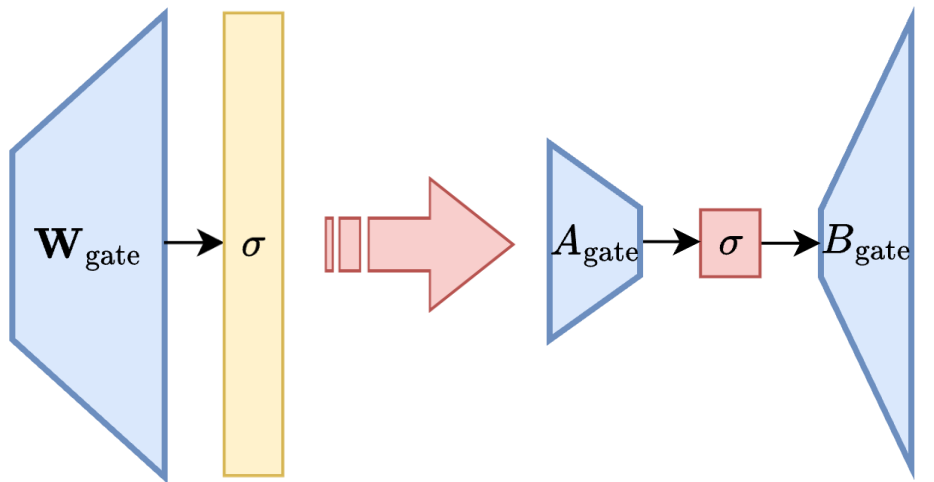


- **Low effective rank for activations**
- **Can we find a principled approach to eliminate the redundancy in activation?**



Proposing CoLA

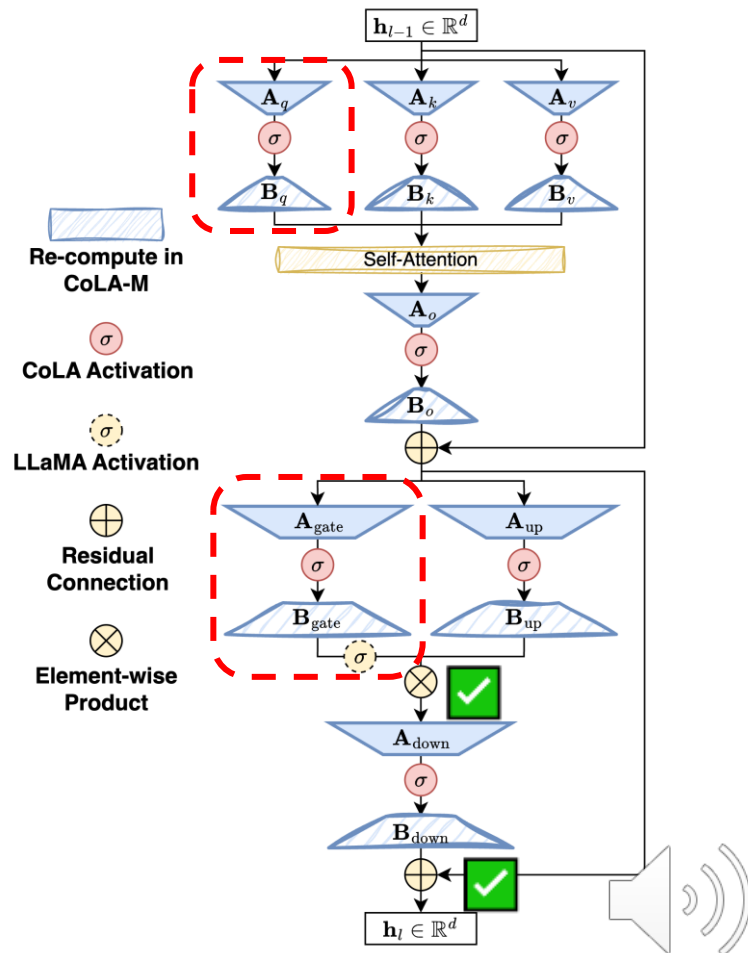
- Low-rank activations via auto-encoders



$$\mathbf{h}_{\text{MLP}} = \sigma(\mathbf{W}\mathbf{x})$$

$$\mathbf{h}_{\text{Linear}} = \mathbf{W}\mathbf{x}$$

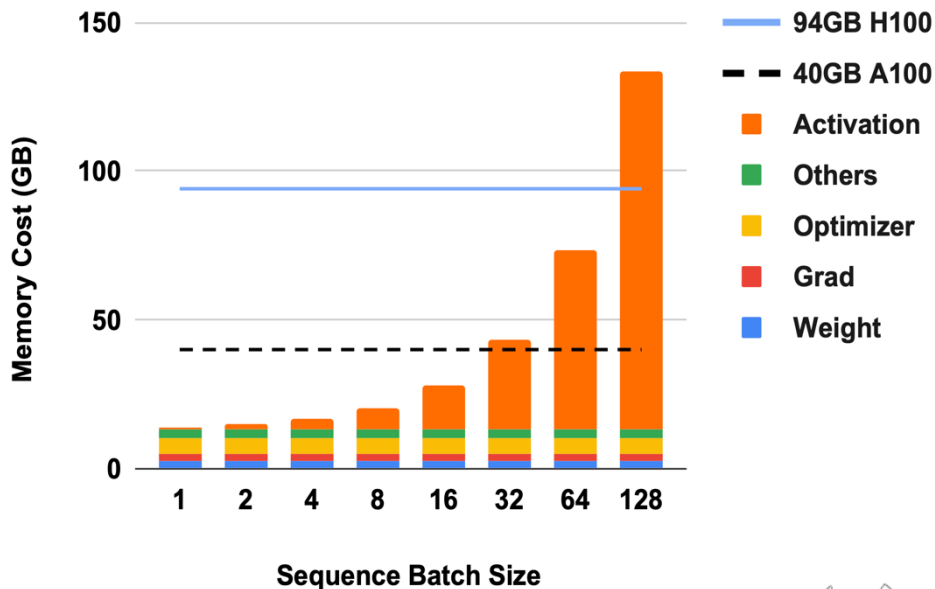
$$\mathbf{h}_{\text{CoLA}} = \mathbf{B} \sigma(\mathbf{A}\mathbf{x})$$



CoLA-M: Pushing Memory Efficiency

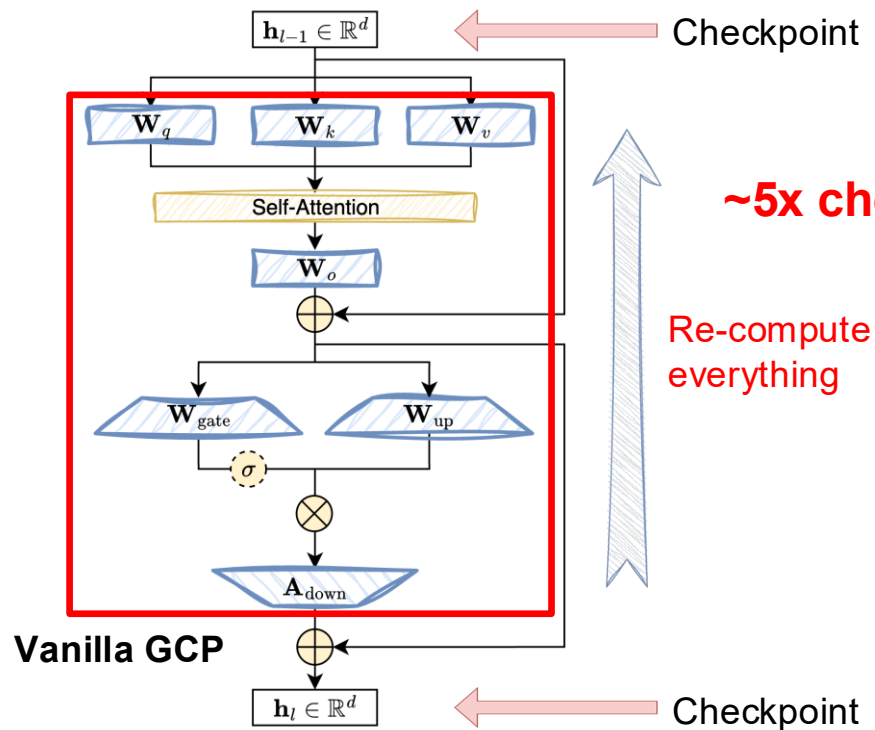
What constitutes memory usage in LLM pre-training?

- **Activation memory can be substantial**
 - Especially when larger batch sizes are used for faster training
- **Gradient checkpointing (GCP) is often used in practice**
 - Introduces significant re-computation overhead for full-rank models
- **CoLA-M achieves efficient gradient checkpointing**



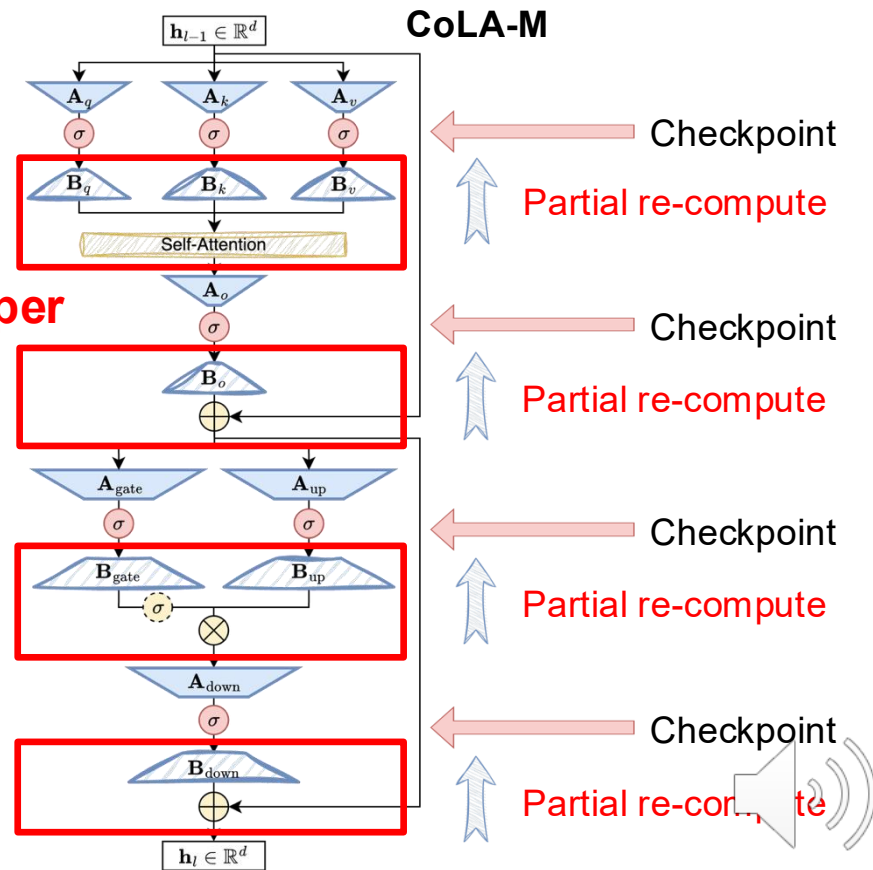
CoLA-M: Pushing Memory Efficiency

CoLA-M only checkpoints at low-rank dimensions



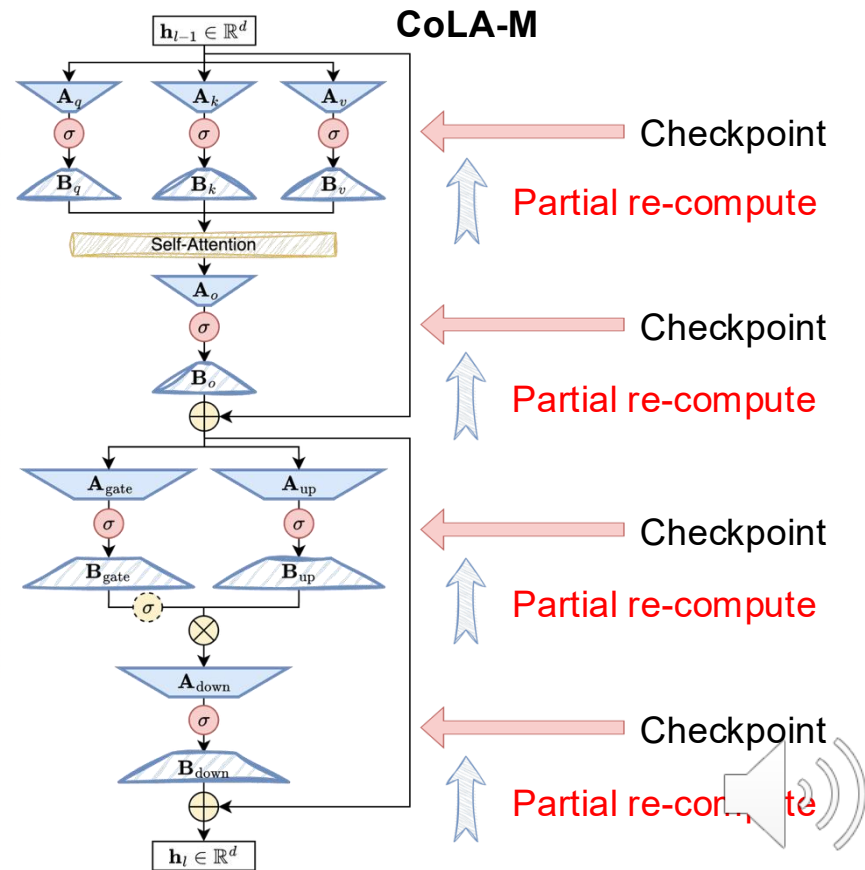
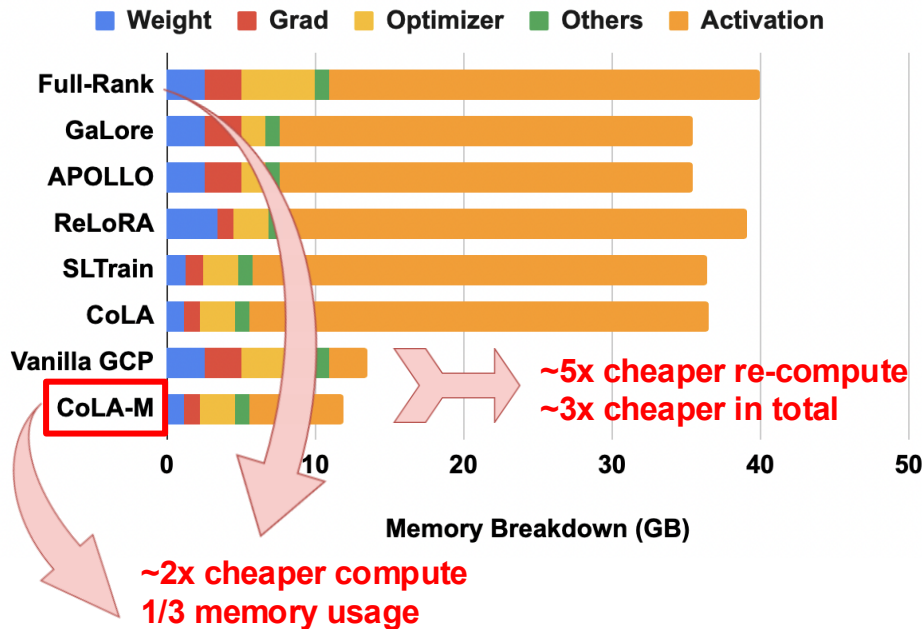
~5x cheaper

Re-compute everything



CoLA-M: Pushing Memory Efficiency

CoLA-M only checkpoints at low-rank dimensions



Experimental Results

- **Pre-training within compute-optimal budget**
 - From LLaMA 60M to 1B, CoLA performs on par with full-rank models while being the smallest

	60M			130M			350M			1B		
<i>r / d</i> <i>Tokens</i>	128 / 512 1.1B			256 / 768 2.2B			256 / 1024 6.4B			512 / 2048 13.1B		
	PPL	Param (M)	Mem (GB)	PPL	Param (M)	Mem (GB)	PPL	Param (M)	Mem (GB)	PPL	Param (M)	Mem (GB)
Full-rank	34.06	58	0.43	24.36	134	1.00	18.80	368	2.74	15.56	1339	9.98
ReLoRA	37.04	58	0.37	29.37	134	0.86	29.08	368	1.94	18.33	1339	6.79
GaLore	34.88	58	0.36	25.36	134	0.79	18.95	368	1.90	15.64	1339	6.60
SLTrain	34.15	44	0.32	26.04	97	0.72	19.42	194	1.45	16.14	646	4.81
CoLA	34.04	43	0.32	24.48	94	0.70	19.40	185	1.38	15.52	609	4.54

Lowest PPL
2x smaller model
1/2 memory used



Experimental Results

- **Pre-training within compute-optimal budget**
 - For LLaMA 7B, CoLA(-M) with BF16 optimizer consumes only ~1/3 memory compared to full-rank/GaLore with 8-bit optimizer
 - CoLA also significantly outperforms baselines in terms of perplexity

	Mem (GB)	10k	40k	80k	120k	150k
8-bit Adam	72.59	N/A	18.09	15.47	14.83	14.61
8-bit GaLore	65.16	26.87	17.94	15.39	14.95	14.65
SLTrain	60.91	27.59	N/A			
CoLA-M	26.82	22.76	16.21	13.82	13.09	12.73

~1/3 memory

Lower perplexity



Experimental Results

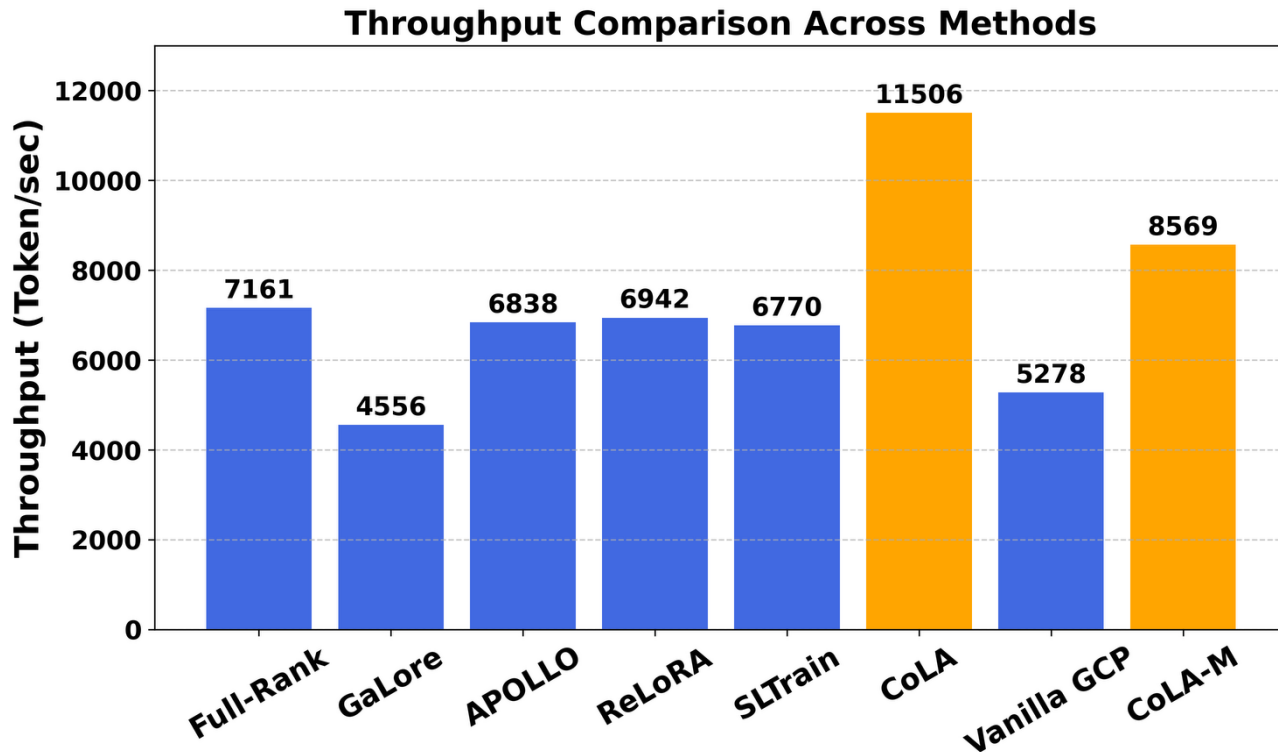
- **Pre-training beyond compute-optimal budget**
 - LLaMA-350M with 51B tokens: CoLA outperforms full-rank LLaMA by **13.96** vs 14.47
 - BERT-350M with 85B tokens: fine-tuned CoLA outperforms full-rank BERT on GLUE by average score **83.5** vs 82.7

	Pre-Training Loss	QQP	SST-2	MRPC	COLA	QNLI	MNLI	RTE	STS-B	GLUE Avg
BERT _{Large}	1.263	91.1	92.1	90.7	53.1	91.6	84.3	69.9	88.9	82.7
CoLA	1.257	91.2	92.3	90.6	54.1	91.7	84.3	74.2	89.7	83.5



Experimental Results

- **Off-the-shelf** system performance



Limitations and Future Ideas

- **CoLA with SOTA distributed training frameworks:** Further scaling up CoLA requires proper model parallelism and is non-trivial for deploying CoLA on to training frameworks such as Megatron-LM and Deepspeed.
- **CoLA under industrial setting:** Due to limited resources, we can only evaluate industrial setting for BERT level models. For over billion scale decoder-only auto-regressive models trained with trillions of tokens, CoLA's effectiveness still needs validations.
 - We are open to collaboration with industry for validating large scale CoLA training
- **CoLA with further improvements:** There are various modifications and improvements can be made to further improve CoLA's efficacy and efficiency. Follow up works such as LaX (NeurIPS 2025)[1], LOST[2], CR-Net[3] have demonstrated this.

[1] Zhang, Ruijie, et al. "LaX: Boosting Low-Rank Training of Foundation Models via Latent Crossing." *arXiv preprint arXiv:2505.21732* (2025).

[2] Li, Jiaxi, et al. "Lost: Low-rank and sparse pre-training for large language models." *arXiv preprint arXiv:2508.02668* (2025).

[3] Kong, Boao, et al. "CR-Net: Scaling Parameter-Efficient Training with Cross-Layer Low-Rank Structure." *arXiv preprint arXiv:2509.18993* (2025).



Thank you for listening!

Q & A

